INSIDE SKILLS

Agent Skills and the skills CLI

Installing, managing, and authoring portable instructions for coding agents

Covers `skills` v1.7.0 · commit `7407f38`

An unofficial guide, generated as a worked example of the `inside-mac` documentation skills

Contents

	Preface	**4**
	About This Guide	4
	Who Should Read This Guide	4
	What to Read	4
	Version and Commit Covered	5
	Conventions	5
	About This Project	6
Chapter 1	**Introduction to Agent Skills**	**7**
	About Agent Skills	7
	A Worked Example	9
	Summary	10
Chapter 2	**Skill Packages**	**11**
	About Skill Packages	11
	Using Skill Packages: Resolving Sources	14
	Versioning and Updates	16
	Skill Packages Reference	16
	Summary	16
Chapter 3	**Installing and Managing Skills**	**18**
	About Installing Skills	18
	Using: Add a Skill	19
	Using: Choose Skills, Agents, and Scope	21
	Using: List, Update, and Remove	21
	Failure and Recovery	23
	Summary	24
Chapter 4	**Agent Targets**	**25**
	About Agent Targets	25
	Agent Directory Reference	27
	Summary	29
Chapter 5	**Authoring and Publishing Skills**	**31**
	About Authoring Skills	31
	Using: Scaffold a New Skill	32
	Using: Write an Effective SKILL.md	32
	Using: Test Locally Before Publishing	33

Using: Publish to GitHub 34
Grouping Skills as a Claude Code Plugin 34
Summary 35

Chapter 6 **Command Reference** **37**
Shared Rules 37
Global Options and Environment 38
`skills add` 39
`skills use` 40
`skills list` 41
`skills remove` 41
`skills find` 42
`skills update` 43
`skills init` 43
`skills experimental_install` and `skills experimental_sync` 44
Summary 44

Appendix A **Glossary** **46**

Appendix B **File Formats** **48**
The Project Lock File: `skills-lock.json` 48
The Global Lock File: `~/.agents/.skill-lock.json` 49

Appendix C **Known Gaps and Open Questions** **51**

Preface

About This Guide

`skills` is an open source command-line tool, published to npm as the `skills` package and run as `npx skills …`. It installs, updates, and removes **agent skills** — small Markdown packages of instructions, named `SKILL.md`, that extend what a coding agent knows how to do — across more than seventy different agent harnesses, from Claude Code and Codex to Cursor, OpenCode, Amp, and many others. Vercel's Guillermo Rauch has publicly described the project as "the npm of AI skills": a single install command works the same way whether the skill lives on GitHub, GitLab, Azure Repos, a plain HTTPS URL, or a local directory.

This guide explains the model behind the CLI, teaches the common workflows, and specifies the complete command contract, in the style of the classic *Inside Macintosh* series: orient first, teach the common path, then give you a reference precise enough to stop guessing at flags.

Who Should Read This Guide

This guide is for two overlapping audiences:

- **Agent users and platform teams** who want to install shared skills across a project or a whole organization's coding agents, in CI or by hand.
- **Skill authors** who want to package instructions once and make them installable everywhere the CLI targets, instead of hand-copying files into each agent's configuration directory.

It assumes you are comfortable with the command line, Git, and npm's `npx` runner. It does not assume familiarity with any particular coding agent.

What to Read

If you want to…	Read…	You can skip…
Understand what a skill is and why the CLI exists	Chapter 1, Introduction to Agent Skills	Chapters 4–6 on a first pass
Understand the anatomy of a skill package and how sources resolve	Chapter 2, Skill Packages	The command reference

Install, list, update, or remove skills day to day	Chapter 3, Installing and Managing Skills	Chapter 5
Find your agent's exact skills directory	Chapter 4, Agent Targets	Chapters 1–2
Write and publish a new skill	Chapter 5, Authoring and Publishing Skills	Chapter 4
Look up exact flags, exit statuses, or files written	Chapter 6, Command Reference	The narrative chapters

Version and Commit Covered

This guide describes `skills` **v1.7.0**, cloned from `https://github.com/vercel-labs/skills` at commit `7407f3893ad4dceab546ac002c3ef806e4000c73` (short form `7407f38`). Every command transcript, file path, and table in this guide was checked against that commit's source and, where shown as a transcript, against real output captured by running the CLI from a clean temporary `HOME` directory. The `skills` project moves quickly; re-check flags against `skills --help` before relying on this guide for a materially newer release.

Conventions

This guide uses a few consistent markers:

> **NOTE**
> Optional context that helps you understand *why* something behaves the way it does. Skipping a note never breaks a procedure.

> **IMPORTANT**
> A condition that changes the outcome of the procedure you are reading. Read these before you run the command.

> **WARNING**
> An action with a real, sometimes destructive, consequence — deleting files, overwriting an install, or sending data over the network. Read the safer alternative before proceeding.

Claims that could not be confirmed directly from the source, tests, or a real command run are labeled **Inferred**. There are three in this guide; each names exactly what is uncertain and why.

Command transcripts show a leading `$` for the invocation and the program's real stdout/stderr below it, trimmed to the lines that matter. Paths use `~` for the user's home directory and `.` for the current project directory, matching how the CLI itself shortens paths in its own output.

About This Project

This is an **unofficial** guide. It is not affiliated with, endorsed by, or reviewed by Vercel, Inc. or the `skills` maintainers. It was produced as a worked example of the `inside-mac` documentation skill set, applied to a real, actively developed open source CLI.

`skills` is copyright its contributors and distributed under the MIT License. Its source, issue tracker, and license text live at `https://github.com/vercel-labs/skills`. All command names, flag names, and short output fragments quoted here are drawn from that public repository; the surrounding explanation is original to this guide.

CHAPTER 1

Introduction to Agent Skills

A coding agent's abilities come from three places: the model, the tools it can call, and the instructions it has been given. A **skill** is a portable unit of the third kind — a folder with a `SKILL.md` file at its root, written so an agent can decide when to read it and what to do once it has. The `skills` CLI (npm package `skills`, run as `npx skills …`) installs, updates, lists, and removes skills across whichever coding agents you use, without you hand-copying files into each agent's own configuration directory.

Read this chapter if you are new to agent skills or to this CLI. **Before you begin**, you should have Node.js and `npx` available; nothing else. **In this chapter** you will learn what a skill is, what problem the CLI solves, how an agent discovers an installed skill at all, and you will walk through one complete install end to end. **You can skip** this chapter if you have already installed skills with this CLI and want to jump straight to Chapter 3 (day-to-day use), Chapter 4 (agent directory reference), or Chapter 6 (command reference).

About Agent Skills

What a Skill Is

A skill is a directory containing a file named `SKILL.md`, plus any supporting resources — scripts, templates, reference documents — that the instructions point to. `SKILL.md` opens with YAML frontmatter delimited by `---` lines, followed by Markdown body text:

```
---
name: web-design-guidelines
description: Review UI code for Web Interface Guidelines compliance. Use when asked
to "review my UI", "check accessibility", "audit design", or "review UX".
---
```

The frontmatter's `name` and `description` are the only fields the CLI requires; a `SKILL.md` missing either one is silently skipped during discovery, with a warning printed to the console. `description` is what an agent (or a human skimming a list) uses to decide *when* to load the skill, so it

does double duty as documentation and as a matching signal — Chapter 5 covers how to write a description that earns its place.

Progressive Disclosure

The instructions in `SKILL.md` are meant to be read completely once a skill is relevant, but the frontmatter `description` should be readable — and distinguishing — without loading the rest of the file at all. This is progressive disclosure applied to agent context: an agent (or the CLI's own `find` and `list` commands) can scan hundreds of descriptions cheaply, then pay the cost of reading a full `SKILL.md` only for the one or two skills a task actually needs. A skill's supporting files extend this further: a `SKILL.md` can defer detail to a linked reference document that only gets read if the instructions tell the agent to open it, rather than inlining everything up front.

The Problem the CLI Solves

Every coding agent that supports skills reads them from its own directory, by its own convention — `.claude/skills` for Claude Code, `.agents/skills` for Codex, OpenCode, and dozens of others, `.windsurf/skills` for Windsurf, and so on for more than seventy agents in total. Without a shared installer, distributing one skill to a team using three different agents means either maintaining three copies or hand-symlinking directories. `skills` treats this as a package-management problem: a skill has a *source* (a GitHub repository, a URL, a local path), and the CLI resolves that source, copies the skill into one canonical location, and links or copies it into every agent directory you choose — the same shape of problem `npm` solves for JavaScript dependencies, applied to Markdown instructions instead of code.

> **NOTE**
> Many agents — Codex, Cursor, OpenCode, Amp, and dozens of others — actually read skills from a shared **universal** directory, `.agents/skills` at the project root or `~/.agents/skills` for a global install, rather than an agent-specific one. Chapter 4 has the full directory-by-directory reference; the important idea for now is that the CLI, not you, tracks which agents share a directory and which need their own copy.

Non-Goals

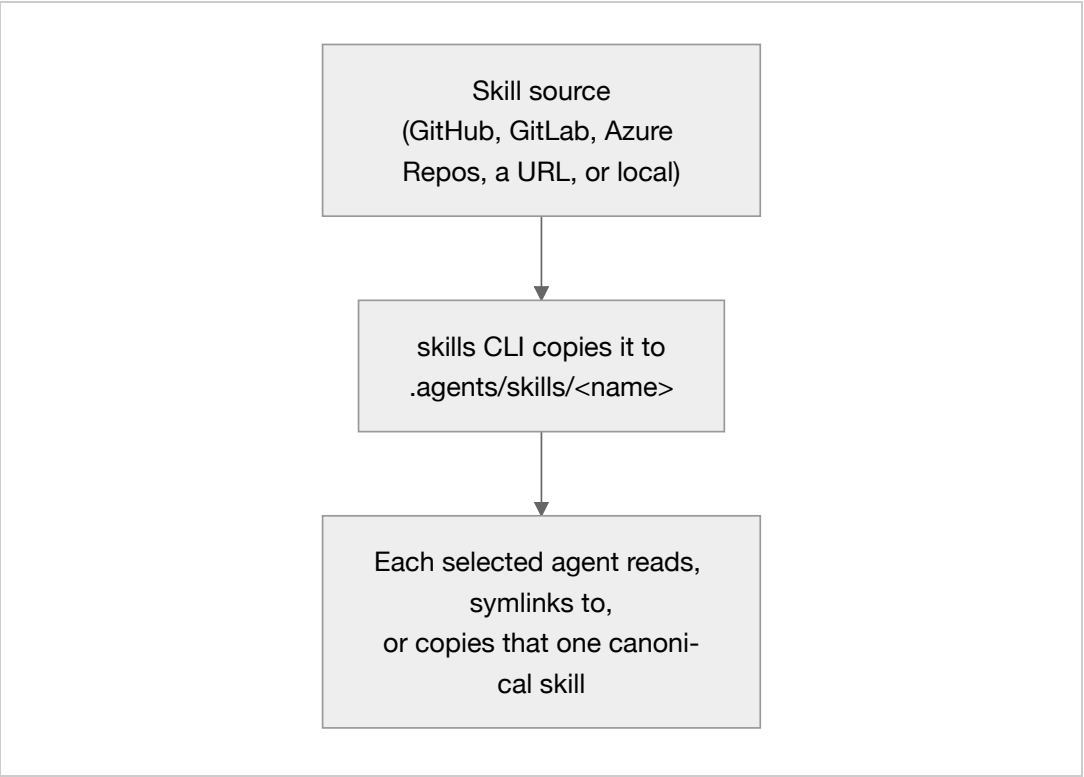
The CLI installs and manages skill *files*. It does not:

- execute a skill or interpret its instructions — that is the agent's job, once the skill is on disk and the agent chooses to read it;
- sandbox or restrict what a skill's instructions can ask the agent to do — the CLI prints a standing reminder that "skills run with full agent permissions" after every install, and means it;
- provide a runtime plugin API, a package registry with its own hosting, or dependency resolution between skills. A skill can bundle supporting files, but skills do not declare dependencies on other skills.

How Agents Discover Skills

Once a skill's files are in an agent's configured skills directory, it is the agent's own harness — not `skills` — that decides when to surface that skill's `description` to the model and when to load the rest of `SKILL.md`. The CLI's job ends at getting the right bytes into the right directory, tracked well enough that a later `update` or `remove` can find them again.

Figure 1-1: A skill's source, the CLI's canonical copy, and each agent's own directory are three distinct locations connected by one installation.



A Worked Example

Installing the `web-design-guidelines` skill from the `vercel-labs/agent-skills` repository, into the current project, for Claude Code only:

```
$ npx skills add vercel-labs/agent-skills --skill web-design-guidelines --agent
claude-code -y
```

Following this one command through the system:

1. The CLI parses `vercel-labs/agent-skills` as GitHub shorthand and resolves it to `https://github.com/vercel-labs/agent-skills.git` (Chapter 2 covers every source format it accepts).
2. It fetches the repository's skill listing — using a fast metadata lookup for eligible GitHub sources, falling back to a shallow `git clone` otherwise — and discovers every `SKILL.md` under the repository's `skills/` directory.
3. `--skill web-design-guidelines` narrows that list to one match; without it, and without `-y`, the CLI would prompt you to choose.
4. It copies that skill's files into the canonical project directory, `.agents/skills/web-design-guidelines/`.
5. Because you asked for one non-universal agent (`claude-code`), it also creates `.claude/skills/web-design-guidelines` as a symlink back to the canonical copy — the default installation mode. `--copy` would write an independent copy there instead.
6. It records the install in `skills-lock.json` at the project root, so a later `skills update` or `skills remove web-design-guidelines` knows exactly where the skill came from.

Claude Code needs no further step: the next time it starts (or reloads its skills), `.claude/skills/web-design-guidelines/SKILL.md` is there to read.

Summary

Mental model. A skill is a `SKILL.md` file with required `name` and `description` frontmatter, plus optional supporting files. The CLI resolves a source, keeps one canonical copy per project or per user, and links or copies that copy into whichever agent directories you choose. Agents decide for themselves when to read an installed skill; the CLI's job stops at installation.

Common path. Resolve a source → discover its skills → choose which skills and which agents → write the canonical copy → link or copy into agent directories → record the install in a lock file.

Invariants.

- A `SKILL.md` without both `name` and `description` in its frontmatter is never installed.
- Skills run with the full permissions of the agent that reads them; the CLI does not sandbox their instructions.
- Universal agents (Chapter 4) always read from `.agents/skills`, never from a separate agent-specific copy, for either scope.

Continue. To understand what makes a valid skill package and how the CLI resolves the sources it can install from, read Chapter 2. To skip straight to day-to-day commands, read Chapter 3.

CHAPTER 2

Skill Packages

A "skill package," in this guide, means whatever the CLI can point at and install from: a single skill directory, or a repository containing many. This chapter specifies the anatomy of both, and the complete set of source formats the CLI accepts when you name one.

Read this chapter if you want to know exactly what makes a `SKILL.md` valid, how a repository organizes more than one skill, or which URL shapes `skills add` understands. **Before you begin**, read Chapter 1 for the one-paragraph model. **In this chapter** you will learn the frontmatter contract, the directory layout the discovery algorithm searches, and every source format from GitHub shorthand to a local path. **You can skip** the source-format reference on a first read and come back to it when you hit a URL shape that does not seem to work.

About Skill Packages

Anatomy of a Skill

At minimum, a skill is one file:

```
web-design-guidelines/
  SKILL.md
```

A richer skill adds supporting files that its instructions reference by relative path:

```
web-design-guidelines/
  SKILL.md
  references/
    contrast-ratios.md
    aria-patterns.md
  scripts/
    check-headings.py
```

The CLI copies a skill's entire directory — everything alongside `SKILL.md` — into its install location, with two exceptions: it always excludes `metadata.json` files and `.git`, `__pycache__`, and `__pypackages__` directories, wherever they appear inside a skill folder. Broken symlinks inside a skill directory are skipped with a warning rather than aborting the whole install.

Frontmatter Fields

`SKILL.md`'s frontmatter is plain YAML between two `---` lines. The parser is intentionally minimal: it supports only the standard `---` YAML delimiter, and explicitly does not support embedded-script frontmatter engines, to avoid executing arbitrary code from a downloaded file.

Field	Required	Type	Meaning
`name`	Yes	string	The skill's identifier. Used to derive its install directory name and to match `--skill` filters.
`description`	Yes	string	What the skill does and when to use it. The only signal most discovery surfaces (agent context, `find`, `list -l`) have before loading the full file.
`license`	No	string	Preserved as-is; notably passed through when a skill installs into Eve, one of the few agents that reads structured frontmatter beyond name/description.
`version`	No	string or number	Preserved as-is; not interpreted by the CLI's own update logic, which tracks content hashes instead (Chapter 6, `skills update`).
`compatibility`	No	any	Preserved as-is for agents (currently Eve) that read it; the CLI itself does not filter on it.
`metadata`	No	object	Free-form. One key has CLI-specific meaning: `metadata.internal: true` marks a skill hidden from discovery by default (below).

> **IMPORTANT**
> `name` and `description` must both be present **and both be strings**. A `SKILL.md` where either field is missing, or where YAML parsed it as a number or boolean, is skipped during discovery with a warning printed to the console — it never silently installs with a blank name.

Internal Skills

A skill whose frontmatter sets `metadata.internal: true` is excluded from ordinary discovery — it will not appear in `skills add <repo> --list`, in an unfiltered `skills add <repo>` skill picker, or in `find` results. It still installs when named explicitly: by `--skill <exact-name>`, by the `owner/repo@skill-name` shorthand (Chapter 2's source-format table), or globally for every disc-

overy call when the environment variable `INSTALL_INTERNAL_SKILLS=1` is set. Repository maintainers use this to ship skills meant for one workflow (an internal test fixture, a skill still under review) without surfacing them to every consumer of the repository.

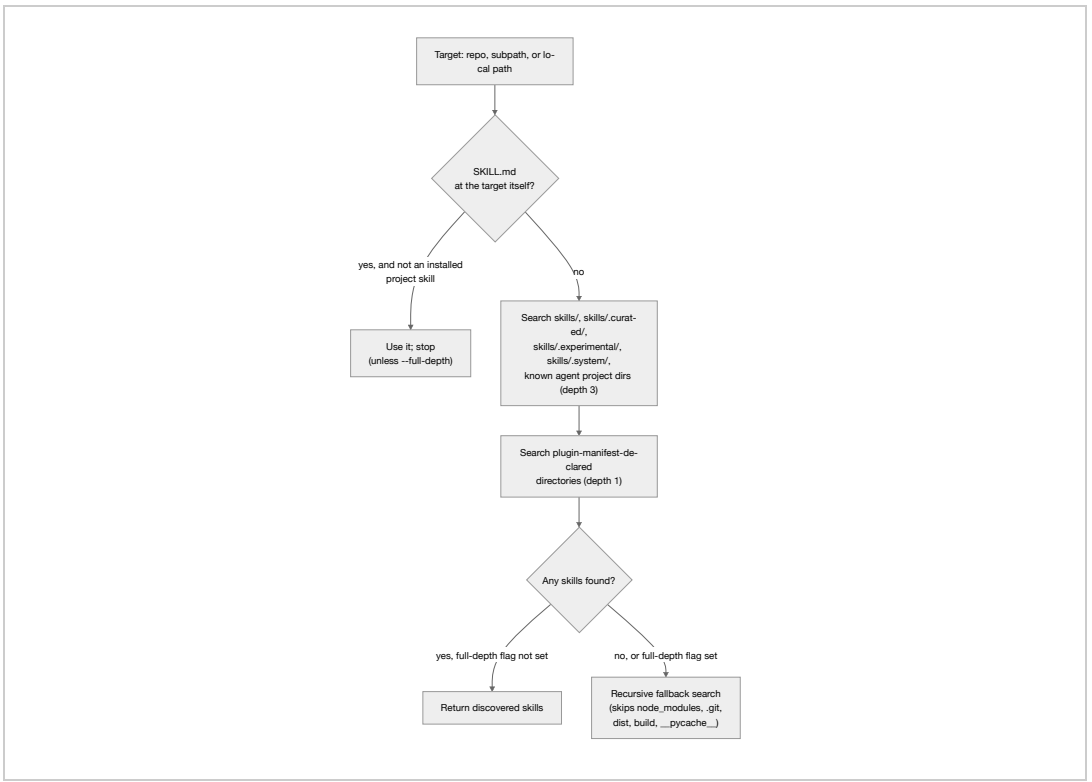
Directory Layout of a Skills Repository

A repository can hold one skill at its root, or many skills nested under conventional container directories. When you point `skills add` at a repository, subpath, or local directory, it searches in this order and stops at the first match unless `--full-depth` is given:

1. **Root `SKILL.md`.** If the target itself has a `SKILL.md`, that single skill is used and the search stops — unless it is a project skill already tracked by `skills-lock.json`, in which case it is skipped so a repository can also serve as a *consumer* of installed skills without the CLI mistaking its own install for a source skill.
2. **Conventional containers**, searched up to three directories deep: `skills/`, `skills/.curated/`, `skills/.experimental/`, `skills/.system/`, and every directory in the table of agent project skill directories from Chapter 4 (so a repository that is itself an agent's project — with its own `.claude/skills/`, say — is still searchable).
3. **Claude Code plugin manifests.** If a `.claude-plugin/marketplace.json` or `.claude-plugin/plugin.json` file declares skill paths or a plugin root, those declared directories are added to the search, and skills found under them are grouped under the plugin's name in `--list` and `list` output.
4. **A full recursive fallback**, only if nothing was found by the steps above, or always when `--full-depth` is passed.

Every discovered skill is deduplicated by `name`; the first occurrence found by the search order above wins. `node_modules`, `.git`, `dist`, `build`, and `__pycache__` directories are never descended into.

Figure 2-1: Discovery walks conventional containers before falling back to a full recursive search.



Using Skill Packages: Resolving Sources

`skills add`, `skills use`, and `skills find` all accept the same source formats, parsed by one shared resolver. Knowing the exact shapes saves a trip to `--help` when a URL you pasted does not behave as expected.

You write	The CLI resolves it as
`owner/repo`	GitHub shorthand → `https://github.com/owner/repo.git` (or another host, honoring `GH_HOST`)
`owner/repo/path/to/skill`	The same repository, narrowed to one subpath
`owner/repo@skill-name`	The same repository, pre-filtering to the skill named `skill-name`
`github:owner/repo`	Identical to `owner/repo` shorthand

`https://github.com/owner/repo`	A full GitHub URL
`https://github.com/owner/repo/tree/<ref>/<path>`	A specific branch or tag, optionally scoped to a subpath
`https://gitlab.com/owner/repo` and `.../-/tree/<ref>/<path>`	The equivalent GitLab forms
`gitlab:owner/repo`	Shorthand for `https://gitlab.com/owner/repo`
`https://dev.azure.com/org/project/_git/repo`	Azure Repos (Cloud or Server); `?path=` and `?version=GBbranch|GTtag` query parameters map to a subpath and ref
`git@host:owner/repo.git`, `ssh://git@host/owner/repo.git`	A direct SSH Git URL, cloned as-is
Any other `https://…git` or bare Git URL	A generic Git source, cloned with `git clone --depth 1`
`./local/path`, `../local/path`, `/absolute/path`	A local directory, read directly with no network access
`https://raw.githubusercontent.com/…`, a GitHub release/archive asset, or a bare `.tar.gz`/`.zip` URL	A direct download: fetched, and if it is an archive, extracted before discovery runs
Any other `http(s)://` URL	A **well-known** source: the CLI first checks `/.well-known/agent-skills/index.json` (falling back to the legacy `/.well-known/skills/index.json`) at that URL's origin, then falls back to treating the URL itself as a direct download
A `#ref` fragment on a Git-like URL, e.g. `owner/repo#v2`	Pins the branch, tag, or full 40-character commit SHA; `#ref@skill-name` combines a ref with a skill filter

> **NOTE**
> A trailing `@skill-name` and a `#ref@skill-name` fragment both set the same internal skill filter; if `--skill` is also given and disagrees, the explicit flag value is merged in rather than silently overridden.

Subpaths are sanitized against path traversal: a subpath containing a `..` segment that would resolve outside the repository root is rejected outright, whether it came from a URL, a `#fragment`, or an Azure Repos `?path=` query parameter.

`skills add` also accepts a Notion workspace or page URL as a source, resolving it through a separate Notion-specific integration rather than the generic Git/HTTP resolver described above. That path is out of scope for this guide's depth of coverage; treat it as an additional supported source

family rather than a fully specified one here — **Inferred**: the exact scope of Notion-source behavior (authentication, pack selection UI) was not verified beyond confirming the source type is recognized and routed separately.

Private Repositories

The same command works for public and private sources. For GitHub HTTPS and shorthand sources, the CLI tries, in order: your existing Git credential helper, then (if the failure looks like a SAML SSO or auth block and the GitHub CLI is authenticated) `gh repo clone`, then an SSH fallback using your loaded SSH keys. It never runs `gh auth token` or copies a GitHub CLI credential into its own process. Setting `GITHUB_TOKEN` or `GH_TOKEN` explicitly enables faster GitHub API-based installs and is required for private-repository update checks that rely on the Trees API rather than a full clone.

Versioning and Updates

A skill's frontmatter `version` field is preserved on disk but is not what the CLI checks to decide whether an update is available. Instead, `skills update` compares a content hash — a GitHub tree SHA for global installs from GitHub, or a SHA-256 over every file in the skill's folder for local-lock project installs and other sources — against the hash recorded at install time. Chapter 6's `skills update` reference has the complete contract, including what happens when a locked skill's upstream path moves or the skill disappears entirely.

Skill Packages Reference

The frontmatter field table and discovery order above (Figure 2-1) are this chapter's canonical contract; there is nothing to add beyond one internal detail worth naming explicitly. A resolved source carries one of six type tags, visible in lock files and JSON output: `github`, `gitlab`, `git`, `local`, `well-known`, or `download`. `skills list --json` and `skills add --json` both surface this value as `sourceType`.

Summary

Mental model. A skill package is a directory with a valid `SKILL.md` at its root; a skills repository is a directory (or Git repository) that may contain many such packages under conventional container paths or manifest-declared paths. The CLI's source resolver turns almost any way you would naturally reference such a repository — shorthand, full URL, pinned ref, subpath, local path — into one of six internal source types.

Common path. Name a source → the resolver classifies it → the CLI fetches or reads it → discovery walks conventional containers, then falls back to a full search → matching `SKILL.md` files with valid frontmatter become installable skills.

Invariants.

- `name` and `description` are always required, and both must be strings.
- A subpath, fragment ref, or query-parameter path can never resolve outside its source's root.
- `metadata.internal: true` skills are hidden from every discovery surface unless named explicitly or unlocked with `INSTALL_INTERNAL_SKILLS=1`.

Continue. Chapter 3 puts these packages to work: choosing which skills and which agents to install to, day to day.

CHAPTER 3

Installing and Managing Skills

This is the chapter for day-to-day use: adding a skill, choosing where it goes, listing what is installed, updating it later, and removing it cleanly.

Read this chapter if you install, update, or remove skills as part of your normal workflow. **Before you begin**, read Chapter 1's worked example if you have never run `skills add`. **In this chapter** you will install skills interactively and non-interactively, choose project versus global scope, select specific skills and agents, and recover from the failures that actually occur in practice. **You can skip** the troubleshooting table on a first read.

About Installing Skills

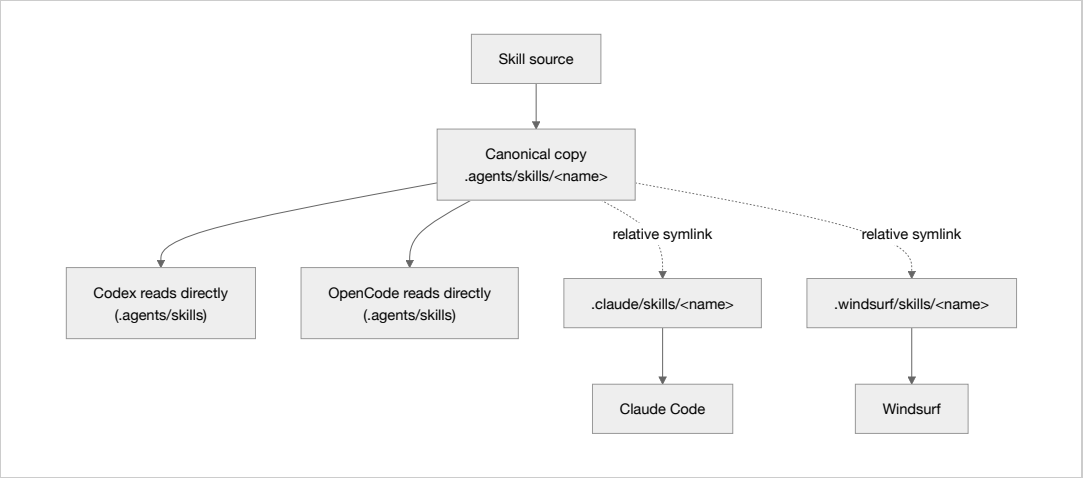
Canonical Copies, Scope, and Install Mode

Every install has three independent choices, each covered in its own section below:

- **Scope** — project (the current directory, the default) or global (your home directory, via `-g` / `--global`).
- **Agents** — which of the CLI's known agents get the skill; auto-detected, prompted for, or given explicitly with `-a` / `--agent`.
- **Install mode** — `symlink` (the default) or `copy` (`--copy`).

Regardless of scope or mode, the CLI keeps one **canonical copy** of each skill's files at `.agents/skills/<name>` (project scope) or `~/.agents/skills/<name>` (global scope). In symlink mode, every **universal** agent (Chapter 4) reads that canonical copy directly with no extra file; every **agent-specific** agent gets a symlink from its own directory back to the canonical copy. In copy mode, every selected agent gets its own independent copy instead, and no canonical directory is used at all for that install. Symlinks are created as relative paths so a project directory stays portable if it is copied or moved as a whole; if symlink creation fails for a target (common on Windows without Developer Mode enabled), the CLI falls back to a real copy for that agent only and reports the fallback rather than failing the whole install.

Figure 3-1: One canonical copy serves every universal agent directly; agent-specific agents get a symlink back to it.



> **IMPORTANT**
> A global install of a **universal** agent never creates that agent's own global directory, even when one is configured (for example, Codex's `~/.codex/skills`). `~/.agents/skills` alone serves every universal agent at both scopes. Only agent-*specific* agents (Chapter 4's non-universal rows) get a real symlink or copy for a global install.

Non-Interactive Detection

When `skills add` runs inside a coding agent's own execution environment — detected via `@vercel/detect-agent` — it behaves as if `-y` were passed and defaults `--agent` to the detected agent plus every universal agent, unless you passed `--agent` yourself. A banner names the detected agent, so this never happens silently:

```
│
●  codex  Agent detected — installing non-interactively
```

Outside of that detection, and without a TTY on stdin, an install that would otherwise need a prompt exits with status 1 and a message pointing at `--agent` and `-y`, rather than hanging or silently doing nothing.

Using: Add a Skill

The minimum command is a source:

```
$ npx skills add vercel-labs/agent-skills
```

Run with no other flags and a real terminal, this: detects installed agents on your machine, asks you to confirm or adjust that agent list, asks whether to install every skill in the repository or pick specific ones (skipped automatically when the repository has exactly one skill), asks project or global scope, and — only when your agent selection spans more than one distinct skills directory — asks symlink or copy, before printing an installation summary and asking you to confirm.

For a scripted or CI install, skip every prompt:

```
$ npx skills add vercel-labs/agent-skills --skill web-design-guidelines --agent
claude-code -g -y
```

`--all` is shorthand for `--skill '*' --agent '*' -y`: every skill in the source, installed for every known agent, without prompts.

JSON Output

`--json` suppresses every decorated prompt, spinner, and note — they move to stderr — and prints exactly one JSON array to stdout on every exit path, success or failure. It requires `-y` or `--all`; `skills add <source> --json` alone (without a non-interactive flag) exits 1 with an explanation rather than hanging on a prompt it cannot show:

```
$ npx skills add ./my-skills --skill demo-skill --agent claude-code --json -y
```

```
[
  {
    "name": "demo-skill",
    "status": "installed",
    "source": "/path/to/my-skills",
    "ref": null,
    "hash": "f96afae8...",
    "path": "/path/to/project/.claude/skills/demo-skill",
    "scope": "project",
    "agents": ["Claude Code"],
    "mode": "copy",
    "security": null
  }
]
```

Each entry's `status` is `installed`, `skipped` (a requested `--skill` name matched nothing), or `failed`. `mode` reflects what actually happened for that skill — `copy` here because a single agent means a single target directory, and the CLI skips the pointless indirection of a symlink to one

place.

Using: Choose Skills, Agents, and Scope

Explicit flags always win over detection or prompts. Agents:

```
$ npx skills add vercel-labs/agent-skills --agent claude-code cursor -y   # named
agents
$ npx skills add vercel-labs/agent-skills --agent '*' -y                  # every
agent
$ npx skills add vercel-labs/agent-skills --agent claude-code -g -y       # global
scope
```

Without `--agent`, the CLI detects which agents are actually configured on your machine (Chapter 4) and, when exactly one is found, installs to it plus every universal agent automatically. With several found and neither `-y` nor `--agent` given, it prompts with a searchable multi-select, pre-populated from your last selection (persisted per machine, not per project).

`--skill` accepts one or more names, matched case-insensitively against a skill's `name` frontmatter — a name with spaces must be quoted as one shell argument:

```
$ npx skills add vercel-labs/agent-skills --skill frontend-design skill-creator
$ npx skills add owner/repo --skill "Convex Best Practices"
```

`--skill '*'` selects every non-internal skill — equivalent to answering "install everything" at the interactive prompt. `--list` / `-l` prints every available skill's name and description without installing anything, a quick way to discover a repository's skill names before writing the `--skill` list you actually want:

```
$ npx skills add vercel-labs/agent-skills --list
```

Using: List, Update, and Remove

`skills list` (alias `ls`) defaults to project scope; `-g` switches to global; `-a <agent>` filters by agent:

```
$ npx skills list
Project Skills

demo-skill  ./.agents/skills/demo-skill
  Agents: Codex  Source: ./my-skills
```

`skills update` (aliases `check`, `upgrade`) checks — and, unless it only finds nothing to do, refreshes — every locked skill whose source supports an update check. It prompts for project, global, or both scope unless `-g`, `-p`, or a positional skill name narrows it, or unless it is running non-interactively, in which case it infers project scope if the project has any locked skills and falls back to global otherwise:

```
$ npx skills update
Checking for skill updates…

✓ Updated 2 skill(s)
```

A skill installed from a local path, a bare Git URL without a recorded subpath, or a private repository the CLI cannot re-query is reported as unable to check automatically, with the exact reinstall command to refresh it by hand — `skills update` never silently gives up on a skill without telling you why.

`skills remove` (aliases `rm`, `r`) accepts names, or opens an interactive multi-select with none:

```
$ npx skills remove demo-skill -y
✓ Successfully removed 1 skill(s)
```

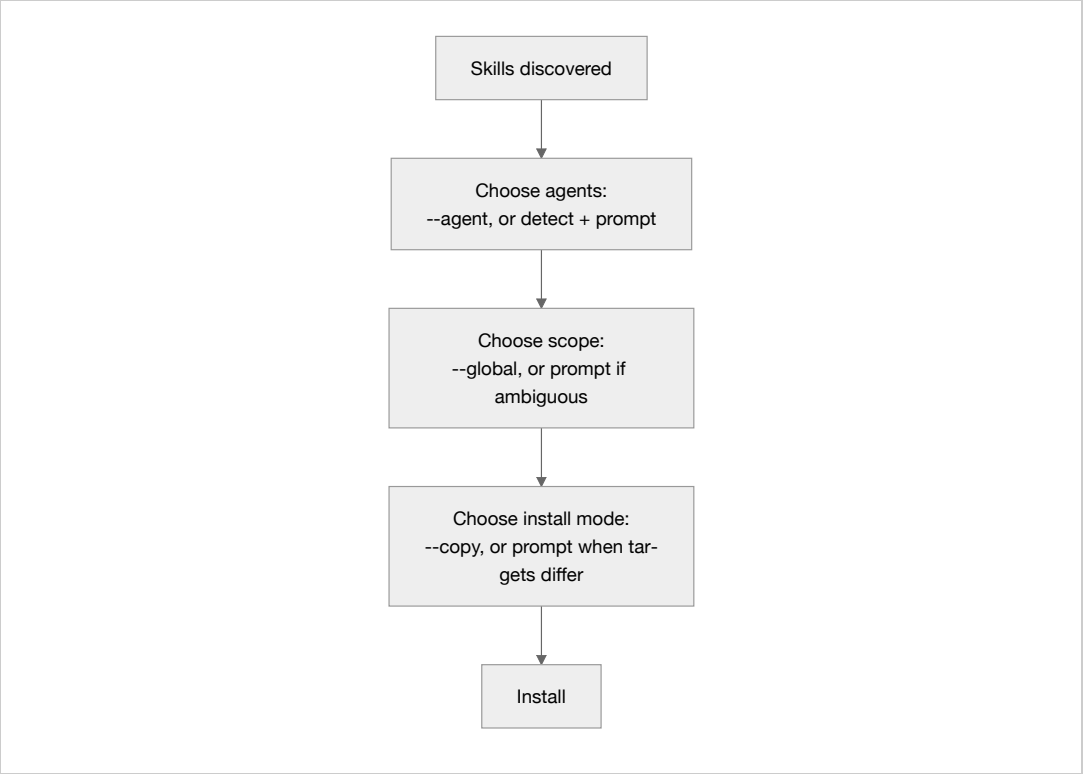
`--all` removes everything and cannot be combined with named skills — the CLI refuses that combination outright, because early versions of this same flag silently discarded the named skills and wiped every install instead:

```
$ npx skills remove --skill foo --all
Cannot combine --all with specific skill names.
Use `skills remove --all` to remove every skill, or omit --all to remove
only the named skills.
Example: skills remove foo -y
```

WARNING

Removing a skill deletes its canonical directory only when no other installed agent still references it, but it always removes the agent-specific copy or symlink for every agent you targeted (or, with no `--agent`, every known agent) — including ghost symlinks left behind by an agent you no longer use. There is no undo short of reinstalling.

Figure 3-2: Choosing agents, scope, and install mode are three independent decisions, in this order.



Failure and Recovery

Symptom	Cause	Recovery
`Missing required argument: source`	`skills add` (or `use`) called with no source	Pass a source; see Chapter 2's format table
Install hangs, then a non-interactive-required error	A prompt was needed but stdin is not a TTY	Add `--agent <name>` (or `'*'`) and `-y`
`No valid skills found`	The source has no `SKILL.md` with both `name` and `description`	Check the source path/subpath; try `--full-depth` if skills are nested unusually deep
Symlinks silently became copies	Symlink creation failed for that target (commonly Windows without Developer Mode)	Enable Developer Mode, or accept `--copy` as the working fallback the CLI already used

Clone times out on a large repository	The default 5-minute clone timeout was exceeded	Set `SKILLS_CLONE_TIMEOUT_MS` higher, or pass a `local` path to an already-cloned copy
Authentication fails for a private repository	Git credentials, `gh`, and SSH all failed in sequence	Confirm access with `gh auth status -h <host>` or `ssh -T git@<host>`, per the error's exact suggestion
`update` reports a skill "cannot be checked automatically"	Installed from a local path, an un-pathed Git URL, or an inaccessible private repo	Run the printed `skills add <source> -g -y` command by hand to refresh it
`remove --skill foo --all` refused	`--all` and named skills were combined	Choose one: `remove --all`, or `remove foo -y`

Summary

Mental model. Every install is scope (project/global) × agents × install mode (symlink/copy), applied to one or more discovered skills, from one resolved source, recorded in a lock file for later update or removal.

Common path. `skills add <source>` → pick skills → pick agents → pick scope → confirm → installed, linked, and locked. `skills list` to check what is there; `skills update` to refresh it; `skills remove` to take it back out.

Invariants.

- `--json` always emits exactly one JSON array to stdout, on every exit path.
- `--all` on `remove` can never be combined with named skills.
- A canonical directory is only deleted when no remaining agent still references it.

Continue. Chapter 4 is the exact directory-by-directory reference for every supported agent; Chapter 6 gives every flag, exit status, and environment variable a canonical entry.

CHAPTER 4

Agent Targets

This chapter is the reference for exactly where the CLI writes files, agent by agent, and how it decides which agents are "installed" on your machine at all.

Read this chapter if you need your agent's exact skills directory, you are debugging why a skill did not appear where you expected, or you are deciding whether to target an agent by name. **Before you begin**, read Chapter 3's install-mode section — this chapter assumes you already know the difference between a canonical copy, a symlink, and a copy. **In this chapter** you will find a directory table covering every universal agent and the most widely used agent-specific ones (79 agents in total; see the note for how to enumerate the rest), how "installed" is detected per agent, and the one genuine special case (Eve subagents). **You can skip** straight to the row for your own agent and treat the rest as reference.

About Agent Targets

Universal Versus Agent-Specific

Chapter 3 introduced two categories of agent. The distinction is entirely mechanical: an agent is **universal** if its configured project `skillsDir` is exactly `.agents/skills`. Every other value makes it **agent-specific**. This single check drives two related but distinct behaviors:

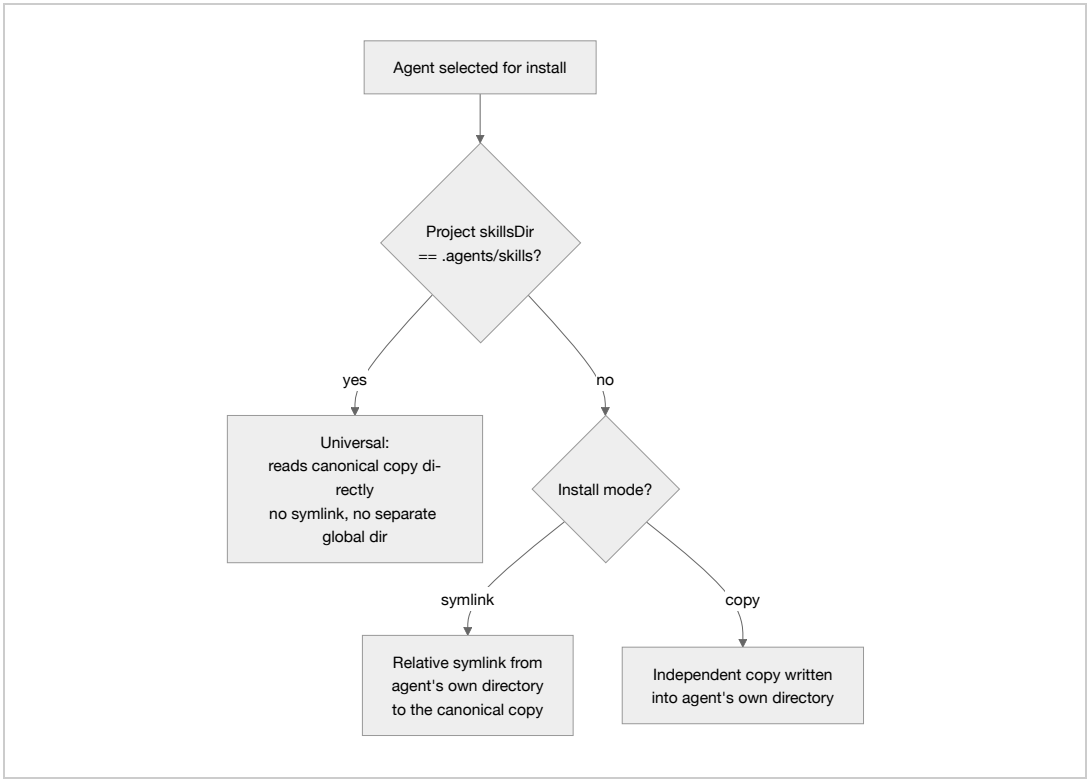
- **Which agents are auto-included and interactively listed.** A slightly narrower set — universal agents minus a small number explicitly flagged `showInUniversalList: false` (Replit, and the CLI's own generic `universal` placeholder target) — is what gets added automatically alongside a detected agent, and shown in the interactive "Universal" locked section.
- **Which agents skip a redundant symlink or agent-specific global directory at install time.** This check is broader: *any* agent whose `skillsDir` is `.agents/skills` — including Replit — is treated as reading the canonical directory directly, so it never gets its own symlink.

> **NOTE**
> In practice this means Replit is invisible in the interactive agent picker and is not auto-added alongside a detected agent, but if you name it explicitly with `--agent replit`, the install behaves exactly like any other universal agent: no per-agent symlink, because `.agents/skills` is already its configured directory.

Detection

`detectInstalledAgents()` runs every agent's own check in parallel and returns whichever return `true`. Detection is deliberately lightweight — almost always "does this configuration directory exist" (`~/.claude` for Claude Code, `~/.cursor` for Cursor) rather than inspecting contents or requiring the agent to be running. A few agents also check the current project — Replit looks for a `.replit` file; Continue, Jazz, AstrBot, and CodeBuddy check both a project and a home directory. It never touches the network.

Figure 4-1: Whether an agent gets its own symlink depends only on its configured project `skillsDir`.



Eve Subagents

Eve is the one agent with structure inside its own skills tree: a project using Eve can define subagents, each with its own skills directory at `agent/subagents/<name>/skills`, alongside the root agent's own `agent/skills`. `skills add --subagent <name>` (repeatable; `root` selects the root agent explicitly) targets one or more of these; `skills list` and `skills remove` both scan every subagent directory they can find under `agent/subagents/` in addition to the root. Eve detection itself requires both an `agent/` directory and an `eve` dependency in the project's `package.json` — the only agent whose detection reads a manifest file rather than checking for a directory alone.

Agent Directory Reference

This version's registry recognizes 79 agents in total. The table below is generated from that registry and lists every **universal** agent (Chapter 4's category that shares `.agents/skills` and therefore needs no directory lookup at all) plus a curated set of the most widely used agent-specific ones — the project-scope `skillsDir` and the global-scope directory used for `-g` / `--global` installs. "Not supported" in the Global column means that agent has no global install location at all — `skills add --global --agent <name>` for it fails outright with an explicit error. Several global paths honor an environment variable override before falling back to the path shown — notably `$CODEX_HOME`, `$CLAUDE_CONFIG_DIR`, `$XDG_CONFIG_HOME`, `$XDG_STATE_HOME`, and several single-purpose `*_HOME` variables listed in Chapter 6's environment reference.

> **NOTE**
> For the complete, always-current list of every valid `--agent` key — including the roughly 40 narrower or region-specific agents not printed below — run any `add` or `remove` command with a deliberately invalid agent name, for example `skills add . --agent x -y`. The CLI's own "Invalid agents" error prints every valid key it knows about, generated from the exact registry your installed version ships, which will not drift out of date the way a printed table can.

Agent	Project `skillsDir`	Global `skillsDir`	Universal
Amp	`.agents/skills`	`~/.config/agents/skills`	Yes
Antigravity	`.agents/skills`	`~/.gemini/antigravity/skills`	Yes
Cline	`.agents/skills`	`~/.agents/skills`	Yes
Codex	`.agents/skills`	`~/.codex/skills`	Yes
Cursor	`.agents/skills`	`~/.cursor/skills`	Yes
Deep Agents	`.agents/skills`	`~/.deepagents/agent/skills`	Yes

Dexto	`.agents/skills`	`~/.agents/skills`	Yes
Droid	`.agents/skills`	`~/.factory/skills`	Yes
Firebender	`.agents/skills`	`~/.firebender/skills`	Yes
Gemini CLI	`.agents/skills`	`~/.gemini/skills`	Yes
GitHub Copilot	`.agents/skills`	`~/.copilot/skills`	Yes
Kilo Code	`.agents/skills`	`~/.kilo/skills`	Yes
Loaf	`.agents/skills`	`~/.agents/skills`	Yes
OpenCode	`.agents/skills`	`~/.config/opencode/skills`	Yes
PromptScript	`.agents/skills`	Not supported	Yes
Sarvam Code	`.agents/skills`	`~/.agents/skills`	Yes
Warp	`.agents/skills`	`~/.agents/skills`	Yes
Zed	`.agents/skills`	`~/.agents/skills`	Yes
Replit	`.agents/skills`	`~/.config/agents/skills`	No (hidden; see note above)
Claude Code	`.claude/skills`	`~/.claude/skills`	No
Windsurf	`.windsurf/skills`	`~/.codeium/windsurf/skills`	No
Roo Code	`.roo/skills`	`~/.roo/skills`	No
Continue	`.continue/skills`	`~/.continue/skills`	No
Goose	`.goose/skills`	`~/.config/goose/skills`	No
AiderDesk	`.aider-desk/skills`	`~/.aider-desk/skills`	No
Devin for Terminal	`.devin/skills`	`~/.config/devin/skills`	No
Junie	`.junie/skills`	`~/.junie/skills`	No
Kiro CLI	`.kiro/skills`	`~/.kiro/skills`	No
Qwen Code	`.qwen/skills`	`~/.qwen/skills`	No
Trae	`.trae/skills`	`~/.trae/skills`	No
ForgeCode	`.forge/skills`	`~/.forge/skills`	No
Grok Build	`.grok/skills`	`~/.grok/skills`	No
Pi	`.pi/skills`	`~/.pi/agent/skills`	No
Eve	`agent/skills`	Not supported	No
OpenClaw	`skills`	`~/.openclaw/skills`	No

Mistral Vibe	`.vibe/skills`	`~/.vibe/skills`	No
OpenHands	`.openhands/skills`	`~/.openhands/skills`	No
Crush	`.crush/skills`	`~/.config/crush/skills`	No

The remaining ~40 agents follow the same two-column shape under their own configuration directory (for example `.zencoder/skills`, `.qoder/skills`, `.trae-cn/skills`) with no further surprises; the note above is the fastest way to enumerate them precisely for the version you have installed.

> **NOTE**
> OpenClaw's global directory additionally depends on which of three possible config directories exists on disk at detection time — `~/.openclaw`, `~/.clawdbot`, or `~/.moltbot`, checked in that order, falling back to `~/.openclaw` if none exist yet. Kilo Code's detection additionally recognizes its legacy `~/.kilocode` directory. Droid deliberately keeps its listed global directory (`~/.factory/skills`) distinct from the canonical `~/.agents/skills` it now installs into, solely so `skills remove` can still clean up skills a previous CLI version placed there directly.

Symlink Versus Copy, Summarized

Install mode	Canonical copy written?	Per-agent result
`symlink` (default)	Yes, at `.agents/skills/<name>` or `~/.agents/skills/<name>`	Universal agents read it directly; agent-specific agents get a relative symlink to it
`copy` (`--copy`, or auto-selected when every target shares one directory)	No	Every selected agent gets its own independent copy

Summary

Mental model. "Universal" is not a special agent — it is what any agent configured to read `.agents/skills` becomes, automatically, because that is already the canonical directory the CLI writes to. Every other agent gets a symlink (or, in copy mode, its own copy) from that canonical location into its own configured directory.

Common path. Look up your agent's row above → know its exact project and global directory → decide whether `--copy` is worth the duplication for your case (Chapter 3 covers when it is: independent per-agent edits, or avoiding symlinks in an environment that mishandles them).

Invariants.

- Any agent with project `skillsDir` equal to `.agents/skills` never gets a separate global directory written during a fresh symlink-mode install, even if one is configured.
- Detection never touches the network and, for most agents, never inspects file contents — only directory existence.

Continue. Chapter 5 is for the other side of this table: writing a skill so it installs cleanly everywhere in it.

CHAPTER 5

Authoring and Publishing Skills

This chapter is for writing a skill, not consuming one: scaffolding a new `SKILL.md`, writing a description worth matching on, testing it locally, and publishing it somewhere the CLI can install it from.

Read this chapter if you are packaging instructions for others (or your future self) to install. **Before you begin**, read Chapter 2's frontmatter contract — this chapter assumes you know what `name`, `description`, and the discovery search order require. **In this chapter** you will scaffold a skill, test it without installing it anywhere, and publish it to a Git host. **You can skip** the plugin-grouping section unless you specifically maintain a Claude Code plugin marketplace.

About Authoring Skills

What Makes a Description Worth Writing

`description` is the entire signal an agent — or a person running `skills find` or `--list` — has before deciding whether to read the rest of `SKILL.md`. A description that only states the topic ("Guidelines for web design") fails to say *when* the skill applies; a description that also states the trigger does not:

```
description: >-
  Review UI code for Web Interface Guidelines compliance. Use when asked to
  "review my UI", "check accessibility", "audit design", or "review UX".
```

Write it the way you would brief a colleague handing off a task: what the skill does, and the concrete phrases or situations that mean it is the right one to reach for. A one-line topic summary is not enough; naming the triggering intent is what makes discovery actually work, whether the "reader" is a model deciding what to load or a person scanning `--list` output.

The Publishing Model

There is no separate registry to push to. A skill is publishable the moment it is a valid `SKILL.md` reachable by one of Chapter 2's source formats: most often, committed to a Git repository. `skills.sh` is a discovery index layered on top of public GitHub repositories — `find` searches it, and

installing from a public repository is what makes a skill appear there — but it is not a separate build or submission step.

Using: Scaffold a New Skill

`skills init [name]` creates a starting `SKILL.md`. With a name, it creates `<name>/SKILL.md`; without one, it writes `SKILL.md` directly into the current directory, using the directory's own name as the skill name:

```
$ npx skills init web-design-guidelines
Initialized skill: web-design-guidelines

Created:
  web-design-guidelines/SKILL.md
```

It refuses to overwrite an existing `SKILL.md` at the target location, printing "Skill already exists" instead — safe to run again by accident. The scaffold it writes is intentionally minimal:

```
---
name: web-design-guidelines
description: A brief description of what this skill does
---

# web-design-guidelines

Instructions for the agent to follow when this skill is activated.

## When to use

Describe when this skill should be used.

## Instructions

1. First step
2. Second step
3. Additional steps as needed
```

Treat `description` as the one line you must not leave at its placeholder value; everything else is a starting shape, not a required structure.

Using: Write an Effective SKILL.md

A few checks before you consider a skill done:

- **`name` and `description` are both present and both strings.** Chapter 2 covers exactly what happens if either is missing — the skill is silently excluded from discovery, not installed with a blank value.
- **The description names a trigger, not just a topic**, as above.
- **Supporting files are referenced by relative path** from `SKILL.md`, so they still resolve after the CLI copies the whole skill directory somewhere else.
- **Nothing sensitive is bundled alongside the skill.** The CLI reminds every installer that skills run with full agent permissions; treat a skill's instructions the same way you would treat a script you are asking someone else to execute, because that is effectively what they are.
- **If the skill should stay hidden from casual discovery**, set `metadata.internal: true` (Chapter 2) rather than relying on obscurity — an internal skill is still installable by exact name.

Using: Test Locally Before Publishing

Two commands let you exercise a skill before it is anywhere public.

`skills add <local-path> --list` discovers and prints every skill under a local directory without installing anything — the fastest way to confirm the CLI actually finds a new skill and parses its frontmatter correctly:

```
$ npx skills add ./my-skill --list
Available Skills
  web-design-guidelines
    Review UI code for Web Interface Guidelines compliance. ...
```

`skills use <local-path>` goes one step further: it materializes the skill into a temporary directory and prints exactly the prompt an agent would receive, without installing it into any agent's directory at all — useful for checking the instructions read the way you intend before anyone installs them:

```
$ npx skills use ./my-skill
You are being given a Skill to execute for the user's next request.

Use the following SKILL.md as your instructions:

<SKILL.md>
---
name: web-design-guidelines
description: ...
---
...
</SKILL.md>
```

Piped into a supported agent's own CLI, or run with `--agent claude-code` or `--agent codex` to launch that agent interactively with the prompt already loaded, this is a full dry run — nothing is written to any skills directory, so there is nothing to remove afterward.

Using: Publish to GitHub

Once a skill is committed to a public GitHub repository, it installs with GitHub shorthand immediately:

```
$ git push origin main
$ npx skills add <owner>/<repo>
```

`skills init`'s own closing output states this directly, plus the URL-hosted alternative for a single file without a full repository:

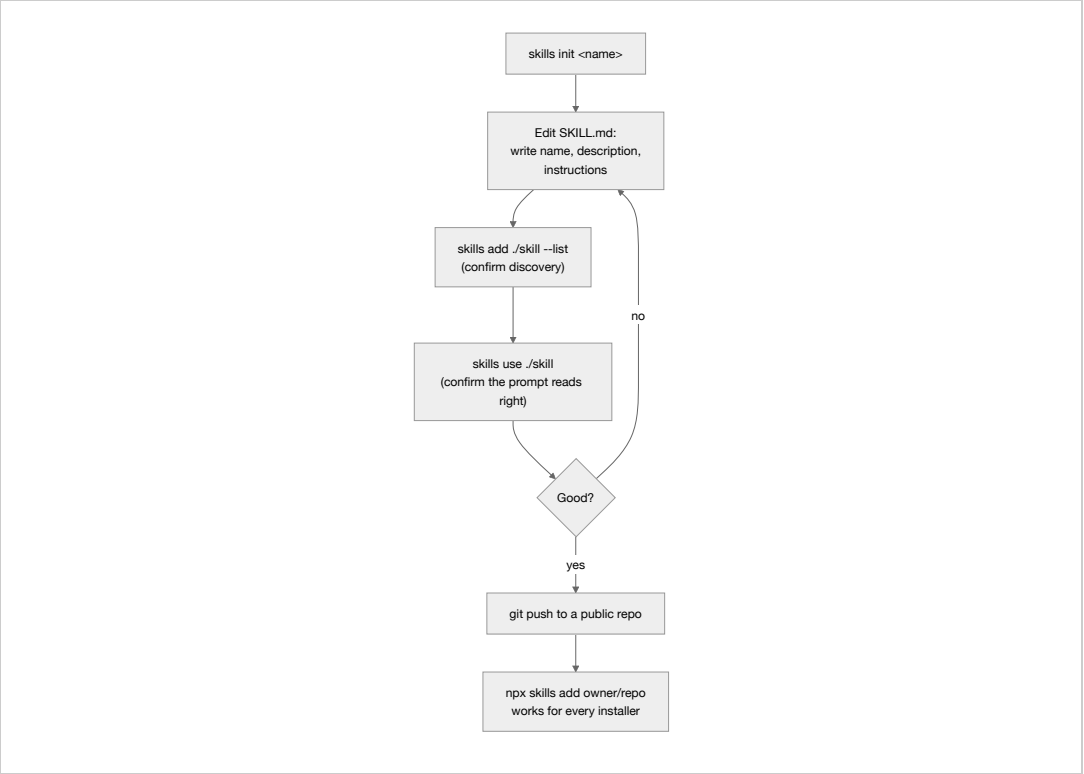
```
Publishing:
  GitHub:  Push to a repo, then npx skills add <owner>/<repo>
  URL:     Host the file, then npx skills add https://example.com/<name>/SKILL.md
```

A repository with more than one skill needs no extra manifest for the CLI to find them — Chapter 2's discovery order already searches a conventional `skills/` directory automatically. `skills add <owner>/<repo> --list` is the fastest way to confirm what a freshly pushed repository actually exposes to installers before telling anyone to run `skills add` against it.

Grouping Skills as a Claude Code Plugin

If your skills repository is also a Claude Code plugin — with a `.claude-plugin/plugin.json` or a multi-plugin `.claude-plugin/marketplace.json` — the CLI's discovery reads the same manifest. Skills whose paths are declared under a named plugin are grouped under that plugin's name in `--list` and `list` output, which is purely a display grouping: installation, agent selection, and the lock file all treat each skill individually regardless of which plugin it was grouped under.

Figure 5-1: Authoring is a loop between the scaffold, local testing, and one publish step; nothing here is a separate submission or review process.



Summary

Mental model. Authoring is: scaffold, write a description that names its trigger, test with `--list` and `use` (which never install anything), then push. There is no build step and no separate registry submission — a reachable, valid `SKILL.md` is a publishable skill.

Common path. `skills init <name>` → edit `SKILL.md` → `skills add ./<name> --list` to confirm discovery → `skills use ./<name>` to confirm the generated prompt → push to a public repository → `skills add owner/repo` from anywhere.

Invariants.

- `skills init` never overwrites an existing `SKILL.md`.
- `skills add --list` and `skills use` never write to any agent's skills directory.
- A skill runs with the full permissions of whatever agent reads it; there is no sandboxing layer to rely on instead of reviewing what you publish.

Continue. Chapter 6 is the exact reference for every command used in this chapter and the two before it — synopsis, every option, exit statuses, and the files each command writes.

CHAPTER 6

Command Reference

This chapter specifies every command's contract: synopsis, arguments, options, precedence, stdio behavior, exit statuses, side effects, idempotency, and examples. Read Chapters 1–5 first for the model; read this chapter to stop guessing at a flag.

Shared Rules

These apply to every command below unless a command's own entry says otherwise.

Binary names. The package installs two identical binaries: `skills` and `add-skill`. Every example here uses `skills`.

Aliases. `add` also answers to `a`, `i`, `install`. `remove` also answers to `rm`, `r`. `find` also answers to `search`, `f`, `s`. `update` also answers to `upgrade`, `check`. `list` also answers to `ls`.

Non-interactive detection. Every command that would otherwise prompt checks, once per process, whether it is running inside a detected coding agent (Chapter 3). If so, it behaves as though `-y` were passed and prints a one-line banner naming the detected agent; it never prompts silently and never hangs waiting on a TTY that will not respond.

`--help`/`-h` short-circuits to that command's help before any side-effecting work runs. **`--version`/`-v`** (first argument only) prints the installed version, e.g. `1.7.0`, and exits 0.

Telemetry. Every command except `init` sends a fire-and-forget usage event (command, source, skill/agent counts — never skill contents) unless `DISABLE_TELEMETRY` or `DO_NOT_TRACK` is set. Failures are always silent and never change a command's exit status; the process waits up to 5 seconds for pending telemetry before exiting.

Exit status convention, used throughout this chapter: `0` means the command completed, including "nothing to do" and a deliberate interactive cancel. `1` means a validation error, a missing prerequisite (TTY, matching skill, matching agent), or at least one real failure inside an otherwise completed batch.

Global Options and Environment

Option	Applies to	Effect
`--help`, `-h`	Every command	Print help, exit 0, no side effects
`--version`, `-v`	Top level	Print the version, exit 0
`-y`, `--yes`	`add`, `remove`, `update`, `experimental_sync`	Skip prompts; use this guide's documented default per command
`-g`, `--global`	`add`, `remove`, `list`, `update`	Operate on `~/.agents/skills` and agents' global directories
`--json`	`add`, `list`	Emit one JSON value to stdout; move decorated output to stderr

Variable	Read by	Effect
`GITHUB_TOKEN`, `GH_TOKEN`	Source resolution, updates	Explicit GitHub API token; optional when Git/gh/SSH auth already works
`GH_HOST`	Source parsing	GitHub Enterprise host for shorthand sources; invalid values fall back to `github.com`
`CODEX_HOME`, `CLAUDE_CONFIG_DIR`, `VIBE_HOME`, `HERMES_HOME`, `AUTOHAND_HOME`, `GROK_HOME`, `SARVAM_HOME`	Per-agent config	Override that agent's default home directory (Chapter 4)
`XDG_CONFIG_HOME`	Several agents' global directories	POSIX config-home base; falls back to `~/.config`
`XDG_STATE_HOME`	The CLI's own global lock file	Lock path becomes `$XDG_STATE_HOME/skills/.skill-lock.json`
`INSTALL_INTERNAL_SKILLS`	Discovery	`1`/`true` includes internal skills (Chapter 2) in every discovery call
`SKILLS_CLONE_TIMEOUT_MS`	`git clone`	Overrides the default 300000 ms clone timeout
`SKILLS_DOWNLOAD_MAX_BYTES`, `SKILLS_EXTRACT_MAX_BYTES`, `SKILLS_EXTRACT_MAX_FILES`	Downloads/archives	Override the default 10 MiB / 25 MiB / 1000-file caps

`SKILLS_API_URL`	`find`	Overrides the default `https://skills.sh` search API
`DISABLE_TELEMETRY`, `DO_NOT_TRACK`	Telemetry	Disable the usage event and the `add` security-audit lookup
`CI`, `GITHUB_ACTIONS`, `GITLAB_CI`, `CIRCLECI`, `TRAVIS`, `BUILDKITE`, `JENKINS_URL`, `TEAMCITY_VERSION`	Telemetry only	Marks the event as CI; no other behavior change

`skills add`

Resolves a source, discovers its skills, and installs selected skills into selected agents' directories.

Synopsis: `skills add <source> [options]` (aliases `a`, `i`, `install`)

Option	Description
`source` (positional, required)	Any format from Chapter 2
`-g`, `--global`	Install to global scope instead of project
`-a`, `--agent <agents...>`	Target these agents (Chapter 4 keys), or `'*'` for all
`-s`, `--skill <skills...>`	Install these skills by name (case-insensitive), or `'*'` for all non-internal
`-l`, `--list`	Print discovered skills and exit; installs nothing
`-y`, `--yes`	Skip prompts, using this section's default order
`--copy`	Independent copy per agent instead of canonical copy plus symlinks
`--metadata <json>`	Attached only to this install's telemetry event
`--subagent <names...>`	Target Eve subagent directories; `root` selects the root agent
`--all`	Shorthand for `--skill '*' --agent '*' -y`
`--full-depth`	Search every subdirectory, even under a root `SKILL.md`
`--json`	One JSON array on stdout; requires `-y`/`--all`; incompatible with `--list`

Behavior. Agent selection precedence: `--agent` > one auto-detected agent plus universal agents > interactive prompt > all agents (`-y` with none detected). Scope precedence: `--global` > interactive prompt > project default. Full decision tree: Chapter 3, Figure 3-2. Decorated progress goes to stdout (stderr under `--json`); errors always go to stderr. Writes the canonical skill directory or per-agent copies, and the project or global lock file; may create an agent-specific project directory that did not exist, only for agents you selected. Re-running the same command overwrites the previous install for that skill/agent pair rather than merging it — idempotent in on-disk result, aside from lock-file timestamps.

Exit statuses: `0` installed (see per-entry JSON `status` under `--json`), `--list` printed, or a deliberate cancel. `1` missing source; bad `--agent`; no skills discovered; `--skill` matched nothing; `--json` without `-y` / `--all`; `--json` with `--list`; a prompt was needed but stdin is not a TTY.

Examples:

```
$ npx skills add vercel-labs/agent-skills --skill web-design-guidelines -a claude-
code -y
$ npx skills add ./local-skills --list
$ npx skills add owner/repo@my-skill -g -y
```

See also: `use` to try a skill first; `update` to refresh; `remove` to undo.

`skills use`

Resolves one skill and prints the prompt an agent would receive, without installing anything.

Synopsis: `skills use <source>[@<skill>] [options]`

Option	Description
`source` (positional, required)	Any Chapter 2 source; trailing `@<skill>` selects one skill among several
`-s`, `--skill <skill>`	Select the skill; must agree with a source's `@<skill>` suffix if both given
`-a`, `--agent <agent>`	Launch `claude-code`, `codex`, or `sarvam-code` interactively with the prompt, instead of printing it
`--full-depth`	Same meaning as for `add`

Behavior. Without `--agent`, the prompt goes to stdout alone — pipeable into another program (`\| claude`); errors go to stderr. With `--agent`, stdio is inherited by the launched process. Writes the skill's files to an auto-cleaned temporary directory only; never touches any agent's configured skills directory. Fully idempotent: no persisted state.

Exit statuses: `0` prompt printed, or the launched agent exited 0. `1` missing/multiple sources; unsupported `--agent`; ambiguous or no matching skill; the launched agent exited non-zero or was not found.

Examples:

```
$ npx skills use vercel-labs/agent-skills@web-design-guidelines | claude
$ npx skills use vercel-labs/agent-skills --skill web-design-guidelines --agent
codex
```

See also: `add` to install the skill instead of trying it once.

`skills list`

Prints installed skills for one scope. **Synopsis:** `skills list [options]` (alias `ls`)

Option	Description
`-g, --global`	List global instead of project (project is default)
`-a, --agent <agents...>`	Filter to skills linked into these agents
`--json`	Emit a JSON array instead of formatted text

Behavior. Read-only; all output on stdout. **Exit statuses:** `0` always, including empty results; `1` an `--agent` value did not match a known agent.

Examples:

```
$ npx skills list
$ npx skills ls -g -a claude-code --json
```

See also: `remove` to act on what `list` shows.

`skills remove`

Removes installed skills from one or more agents. **Synopsis:** `skills remove [skills...] [options]` (aliases `rm`, `r`)

Option	Description
`skills` (positional)	Names to remove; none opens an interactive multi-select on a TTY
`-g, --global`	Remove from global scope instead of project
`-a, --agent <agents...>`	Only these agents' directories; omitted checks and cleans every known agent
`-s, --skill <skills...>`	Alternative to positional names; `'*'` is a documented synonym for `--all`
`-y, --yes`	Skip the confirmation prompt
`--all`	Remove everything; implies -y; **cannot** combine with named skills

Behavior. Deletes each targeted agent's copy or symlink for the named skills. Deletes the canonical directory only if no remaining installed agent still references it, and removes the matching lock entry only in that same case — so a skill still used elsewhere stays updatable. Removing an

already-absent name is a no-op reported as "no skills found to remove."

> **WARNING**
> There is no dry-run flag. `--all` removes every skill in scope immediately once you accept the prompt (or immediately with `-y`). Run `list` first if you are unsure what is installed.

Exit statuses: `0` completed, or a deliberate cancel. `1` `--all` combined with named skills; an invalid `--agent` value.

Examples:

```
$ npx skills remove demo-skill -y
$ npx skills remove --skill '*' -a cursor
$ npx skills remove --all
```

See also: `list` to see what would be affected first.

`skills find`

Searches the `skills.sh` index. **Synopsis:** `skills find [query] [--owner <owner>]` (aliases `search`, `f`, `s`)

Argument/Option	Description
`query`	Free-text terms; with one, results print and the process exits. Without one, on a TTY, opens an interactive incremental search.
`--owner <owner>`	Restrict to one GitHub owner

Behavior. Non-interactive results print to stdout, one skill per block with its install command and `skills.sh` URL. The interactive UI reads raw keypresses; `Enter` installs the highlighted result exactly as `add` would, `Esc` cancels. Running inside a detected agent, or without a TTY, `find` with no query prints a short tip instead of attempting the interactive UI. **Exit status** is always `0`; zero results or a bad `--owner` print a message rather than failing.

Examples:

```
$ npx skills find typescript
$ npx skills find react --owner vercel
```

See also: `add`, which `find` delegates an interactive install to.

`skills update`

Checks locked skills for upstream changes and reinstalls the ones that changed. **Synopsis:** `skills update [skills...] [options]` (aliases `upgrade`, `check`)

Option	Description
`skills` (positional)	Names to limit the check to; none checks every locked skill in scope
`-g`, `--global` / `-p`, `--project`	Check one scope only; both together mean both
`-y`, `--yes`	Skip the scope prompt and the prompt to delete skills removed upstream

Behavior. Scope, non-interactively (no TTY or `-y`): project if the project has any locked skills, else global. For each changed skill, re-runs the equivalent of `add` against its recorded source, updating its files and lock entry; GitHub sources are checked with a lightweight tree-hash API call first, falling back to a full temporary clone when that is unavailable or the recorded path moved. Nothing changed upstream reports "up to date" and writes nothing.

Exit statuses: `0` every checked skill was current or updated. `1` at least one skill failed to update (set only after every skill was still attempted).

Examples:

```
$ npx skills update
$ npx skills update my-skill -g
```

See also: `add`, which `update` re-runs under the hood.

`skills init`

Scaffolds a new `SKILL.md`. **Synopsis:** `skills init [name]`

With a name, creates `<name>/SKILL.md`; without one, creates `./SKILL.md` using the current directory's own name. Never overwrites an existing `SKILL.md` — reports "Skill already exists" instead, making a repeated run a safe no-op. Always exits `0`.

Examples:

```
$ npx skills init my-skill
$ npx skills init
```

See also: `add --list` to confirm the scaffolded skill is discoverable.

`skills experimental_install` and `skills experimental_sync`

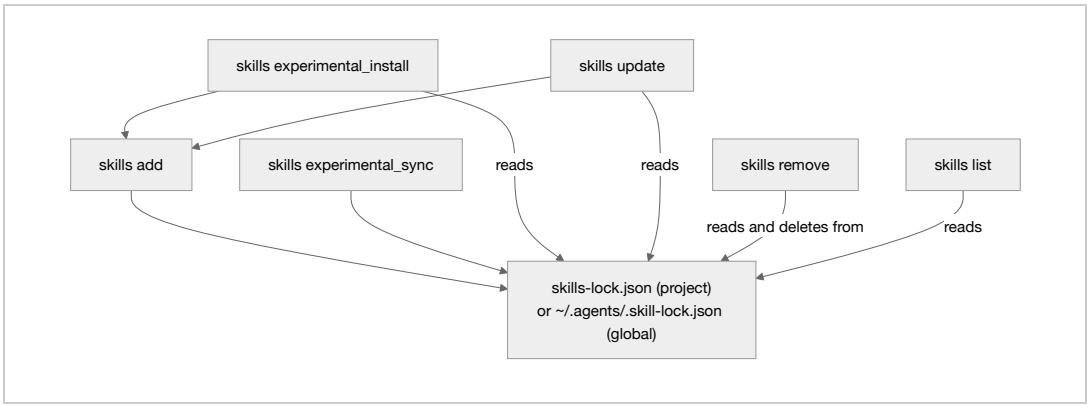
Two project-scope maintenance commands, named `experimental_*` by the CLI itself — their contract is more likely to change in a future release than the commands above.

`skills experimental_install` restores every skill recorded in a project's `skills-lock.json`: it groups locked entries by source and calls the equivalent of `add` for each group, installing only into `.agents/skills` regardless of which agents originally received a symlink or copy. Entries with `sourceType: "node_modules"` are handed to `experimental_sync` instead. Always exits `0`; per-source failures are logged, not fatal to the run. Safe to run repeatedly.

`skills experimental_sync [options]` scans `node_modules` (including scoped `@org/pkg` packages) for `SKILL.md` files at a package's root, `skills/`, or `.agents/skills/`, and installs matches into `.agents/skills` — always symlink mode, always project-scoped — recording each with `sourceType: "node_modules"` and a content hash in `skills-lock.json`. `-a` / `--agent <agents...>` targets specific agents; `-y` / `--yes` skips prompts; `-f` / `--force` reinstalls every discovered skill even if its content hash has not changed. Without `--force`, a later run only reinstalls packages whose skill content actually changed. Always exits `0`.

```
$ npx skills experimental_install
$ npx skills experimental_sync -y --agent claude-code
```

Figure 6-1: Every command reads or writes the same two lock files; only `add`, `experimental_sync`, and `experimental_install` write skill files themselves.



Summary

Command	Writes
`add`	Canonical directory or per-agent copies; project or global lock file
`use`	A temporary directory only, always cleaned up
`remove`	Agent copies/symlinks, conditionally the canonical directory and its lock entry
`update`	Same as `add`, for each skill it refreshes
`init`	One new `SKILL.md` (never overwrites)
`experimental_sync`	Canonical directory (symlink mode only); project lock file
`experimental_install`	Same as `add`, driven by the existing project lock file
`list`, `find` (query mode)	Nothing

Continue. The appendixes that follow collect a glossary, both lock file formats in full, and open questions this guide could not resolve from the source alone.

APPENDIX A

Glossary

agent — A coding tool (Claude Code, Codex, Cursor, and more than seventy others) that reads installed skills from its own configured directory. See Chapter 4.

canonical copy — The one on-disk copy of a skill's files the CLI maintains per scope, at `.agents/skills/<name>` (project) or `~/.agents/skills/<name>` (global). In symlink mode, every other install location either is this copy (universal agents) or points back to it (agent-specific agents). Compare **agent-specific copy**.

agent-specific copy — A skill's files written directly into one agent's own configured directory, either because install mode was `copy`, or because a relative symlink to the canonical copy could not be created and the CLI fell back to a real copy for that agent.

frontmatter — The YAML block at the top of `SKILL.md`, delimited by `---` lines, that carries `name`, `description`, and optional fields. See Chapter 2.

global scope — Installing into the user's home directory (`~/.agents/skills`, or an agent's own global directory) with `-g` / `--global`, available to every project on the machine. Compare **project scope**.

internal skill — A skill whose frontmatter sets `metadata.internal: true`, excluded from ordinary discovery but still installable by exact name or with `INSTALL_INTERNAL_SKILLS=1`. See Chapter 2.

lock file — A JSON file recording which skills are installed and from where, so `update` and `remove` can find them again. There are two: the project-scoped `skills-lock.json` and the global `~/.agents/.skill-lock.json`. See Appendix B.

project scope — Installing into the current project directory (the default), tracked in a `skills-lock.json` meant to be committed alongside the project. Compare **global scope**.

skill — A directory containing a `SKILL.md` file with valid `name` and `description` frontmatter, plus any supporting files its instructions reference. See Chapter 1.

source — Whatever you name to `skills add`, `skills use`, or `skills find`: GitHub shorthand, a full Git/GitLab/Azure Repos URL, a bare Git URL, a local path, or an HTTP(S) URL resolved as well-known or direct-download. See Chapter 2.

universal agent — An agent whose configured project `skillsDir` is exactly `.agents/skills`, so it reads the canonical copy directly with no symlink of its own. See Chapter 4.

well-known source — An HTTP(S) URL that is not a recognized Git host, resolved by checking `/.well-known/agent-skills/index.json` (or the legacy `/.well-known/skills/index.json`) at that URL's origin before falling back to a direct download. See Chapter 2.

APPENDIX B

File Formats

The Project Lock File: `skills-lock.json`

Written at the project root by `add` (project scope), `experimental_sync`, and `experimental_install`; read by `update`, `list`, `remove`, and discovery itself (to avoid re-discovering an already-installed project skill as though it were a source skill — Chapter 2). Meant to be committed to version control: it deliberately omits timestamps so that two branches adding different skills merge cleanly as plain JSON, and its skill entries are always written sorted alphabetically by name for a stable diff.

```
{
  "version": 1,
  "skills": {
    "web-design-guidelines": {
      "source": "vercel-labs/agent-skills",
      "sourceUrl": "https://github.com/vercel-labs/agent-skills.git",
      "sourceType": "github",
      "skillPath": "skills/web-design-guidelines/SKILL.md",
      "computedHash": "9f2b1c...(sha-256 over every file in the skill folder)",
      "ref": null
    }
  }
}
```

Field	Meaning
`version`	Schema version; 1 in this release. An older-versioned file is treated as empty rather than migrated.
`source`	The original source string, or its normalized `owner/repo` form for public GitHub sources
`sourceUrl`	The resolved clone/fetch URL, present when it differs meaningfully from `source` (generic Git and GitLab sources)
`sourceType`	One of `github`, `gitlab`, `git`, `local`, `well-known`, `node_modules`
`ref`	A pinned branch, tag, or commit SHA, if the source specified one

`skillPath`	The skill's `SKILL.md` path relative to the source root; required for `update` to reinstall only this skill rather than the whole source
`computedHash`	SHA-256 over the sorted relative paths and contents of every file in the installed skill folder
`subagents`	For Eve installs, which subagent directories (or the root agent, recorded as `""`) received this skill
`wellKnownDigest`	For well-known sources, a digest used to detect upstream changes without a full content re-fetch

The Global Lock File: `~/.agents/.skill-lock.json`

Written by global-scope `add`, and read by global `update`, `list`, and `remove`. Its path honors `$XDG_STATE_HOME/skills/.skill-lock.json` when that variable is set, falling back to `~/.agents/.skill-lock.json` otherwise. Unlike the project lock file, it does track install and update timestamps, since it is per-machine state, never committed anywhere.

```
{
  "version": 3,
  "skills": {
    "web-design-guidelines": {
      "source": "vercel-labs/agent-skills",
      "sourceType": "github",
      "sourceUrl": "https://github.com/vercel-labs/agent-skills.git",
      "skillPath": "skills/web-design-guidelines/SKILL.md",
      "skillFolderHash": "a1b2c3...(GitHub tree SHA for the skill folder)",
      "installedAt": "2026-01-01T00:00:00.000Z",
      "updatedAt": "2026-01-01T00:00:00.000Z"
    }
  },
  "dismissed": {},
  "lastSelectedAgents": ["claude-code", "cursor"]
}
```

Field	Meaning
`version`	Schema version; 3 in this release. A file at an older version is wiped and rebuilt on next write rather than migrated in place — a deliberately breaking change made when folder-hash tracking was added.
`skillFolderHash`	For GitHub sources, the folder's Git tree SHA, fetched via the Trees API; changes whenever any file under the skill's folder changes upstream
`installedAt` / `updatedAt`	ISO timestamps; `installedAt` is preserved across updates, `updatedAt` refreshes on every successful `add`/`update`
`lastSelectedAgents`	The most recent interactive agent selection, reused to pre-populate the next prompt

`dismissed`	Tracks one-time prompts (currently, whether a `find-skills` follow-up suggestion was dismissed) so they are not repeated

Both lock files use the same skill-name-to-entry shape, and both are best treated as CLI-managed state: hand-editing either is possible (they are plain JSON) but unnecessary for any workflow this guide describes.

APPENDIX C

Known Gaps and Open Questions

This guide was written directly against the `skills` v1.7.0 source, its tests, and real command output captured from that exact commit. A few areas are intentionally underspecified here, either because they are explicitly labeled experimental upstream or because full verification was out of this guide's scope:

- **Notion sources.** `skills add` recognizes and routes Notion workspace and page URLs through a dedicated integration, separate from the Git/HTTP resolver this guide specifies in full (Chapter 2). This guide confirms the source type is recognized but does not specify its authentication or pack-selection behavior in the same depth as every other source format. **Marked Inferred in Chapter 2.**
- **`experimental_install` and `experimental_sync` stability.** Both commands are named `experimental_*` by the CLI itself and are documented here as they behave in v1.7.0; their contract is more likely to change in a future release than the stable `add` / `remove` / `update` / `list` commands.
- **The `skills.sh` discovery index.** `find` and the security-audit annotations shown during `add` both call a hosted API (`https://skills.sh`, `https://add-skill.vercel.sh`) whose own behavior, ranking, and data freshness are outside this guide's scope — this guide documents only the CLI's client-side request and response handling, not the service behind it.
- **Detection heuristics inside `@vercel/detect-agent`.** Chapter 3's non-interactive detection relies on an external package whose exact per-agent environment-variable signals were not individually audited here, beyond the one documented override this CLI applies on top of it (requiring a stronger signal before treating a Cursor terminal session as an active agent run, to avoid false positives in an interactive Cursor terminal).

None of these gaps affect the accuracy of the command reference in Chapter 6, which was checked against the source and, for every command shown with a transcript, against real output.